

GENIE for FASER

A native build without the ATLAS container,
and what changes from 3.04 to 3.06

André Rubbia

October 2026

ETH Zürich, IPA

1. Why the old recipe needed the ATLAS container
2. The new build: one repository, two scripts
3. Running GENIE: `run_genie.py`, from checkout to the FASER simulation
4. Pitfalls solved along the way (macOS / modern ROOT)
5. GENIE 3.04 (FASER fork) → 3.06.02: what changed
6. Validation: 3.06 vs 3.04, Mac vs Ubuntu, and a regression test
7. Next steps

The old recipe

How 3.04 was built

- `setupATLAS -c centos7`: Apptainer container with CentOS 7
- `asetup Athena,22.0.49`: only to get compilers and libraries from CVMFS (gcc 11, ROOT 6.24, Pythia6, LHAPDF, GSL, ...)
- TPythia6 built as an *Athena/Calypso* CMake project
- GENIE itself: plain `./configure && make`

Problems

- CentOS 7 is end-of-life: the container exists only to run old binaries
- mixed gcc8 and gcc11 libraries on one `LD_LIBRARY_PATH`
- broken Pythia6 link line in `Make.include`
- not usable on a Mac
- no easy path to move to Pythia8
- GENIE writes only GHEP: extra `convertGHEP.C` pass needed for the gFaser ntuples

Key point

GENIE contains no ATLAS code. Athena was only a convenient way to get a software stack.

What GENIE actually needs

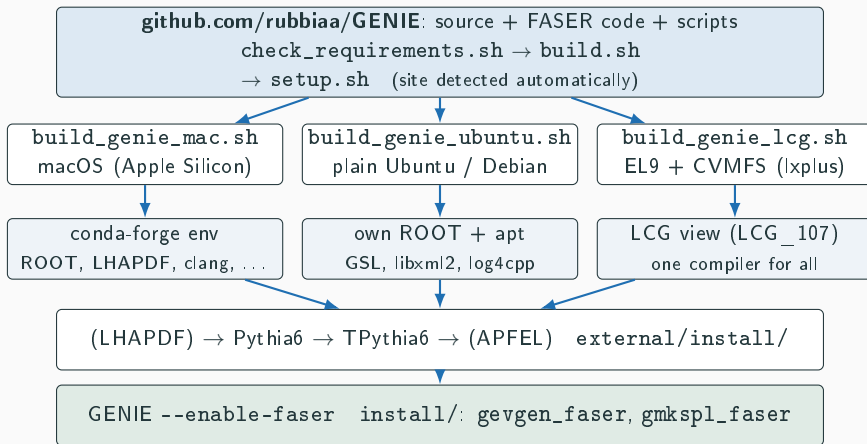
Dependency	Role	Source in the new build
ROOT (Geom, MathMore)	I/O, geometry (TGeo), maths	conda-forge / LCG / own build
Pythia 6.4.28	hadronization, decays	built from alisw/pythia6
TPythia6 (libEGPythia6)	ROOT interface to Pythia6	built here (plain CMake)
LHAPDF 6	PDFs	conda-forge / LCG / built here
GSL, libxml2, log4cpp	numerics, config, logging	conda-forge / LCG / apt
<i>optional</i> : APFEL 3.0.6	HEDIS structure functions	WITH_APFEL=1
<i>optional</i> : Pythia 8	Pythia8 hadronization	WITH_PYTHIA8=1

TPythia6 was removed from ROOT in 6.32, so it is compiled from the old ROOT sources.

APFEL is off by default: the FASER splines and tunes do not use HEDIS.

The new build

Overview



No container, no Athena, no asetup. The same commands on all three platforms.

Repository = work directory

```
GENIE3.06/                <- git repository
|-- README.md
|-- GENIE_3.04_vs_3.06.md
|-- check_requirements.sh
|-- setup.sh  build.sh    <- pick the site
|-- setup_{mac,lcg,ubuntu}.sh
|-- build_genie_{mac,lcg,ubuntu}.sh
|-- data/      GDML/ obsolete/ fluxes/
|-- GenieGenerator/      <- GENIE 3.06.02 + FASER
|  |-- src/  config/  data/
|  |-- faser/    (run scripts, Splines, Ntuple)
|  |  |-- TPythia6_standalone/
|  |  `-- build/external/  (patch, Pythia6 CMake)
|-- install/          \
|-- external/         |  created by the
|-- build/            |  build, ignored by git
|-- faser_xsec/        |
`-- output/           /  <- .ghep/.gfaser
```

- Full GENIE history kept (7 962 commits), FASER work on top
- Build products, downloads, splines and event files are *git-ignored*
- Geometries (data/GDML, data/obsolete) and fluxes (data/fluxes) outside the GENIE tree
- Large Kling 2023 flux ntuples (≤ 575 MB) not in git:
data/fluxes/getFluxNtp.sh
- Scripts find the source tree and work directory automatically

Checking the requirements first

```
$ ./check_requirements.sh
site = ubuntu
== 1) platform prerequisites
[ok]      libgsl-dev      2.7.1
[MISSING] liblog4cpp5-dev apt ...
== 3) compilers, ROOT, libraries
[ok]      ROOT           6.32.02
[MISSING] libMathMore    rebuild ROOT
[TO BUILD] LHAPDF6 lib    external/
== 4) built by build.sh
[TO BUILD] TPythia6       libEGPythia6
[TO BUILD] gevgen_faser    install/bin
== 5) inputs for running
[RUN]      splines        faser_xsec/
RESULT: 2 prerequisite(s) MISSING
```

Per platform, it checks:

1. what you must provide: Xcode + conda env / LCG view / apt packages
2. that `setup_<site>.sh` loads
3. compilers, ROOT (Geom, MathMore, EG), libxml2, log4cpp, LHAPDF 6
4. Pythia6, TPythia6, APFEL, Pythia8, GENIE
5. splines, fluxes, geometry

Status: [MISSING] install it, [TO BUILD] build.sh makes it, [RUN] only to run.

Exit code 0 ready, 1 missing, 2 build, 3 run inputs. build.sh runs it first and stops on 1.

Building on a Mac

```
git clone https://github.com/rubbiaa/GENIE.git GENIE3.06
cd GENIE3.06
./check_requirements.sh      # what is there, what is missing
./build.sh                  # = build_genie_mac.sh, first time ~20 min
source setup.sh              # = setup_mac.sh, every new terminal
```

What the script does:

1. installs a private Miniforge in ~/miniforge3 (does not touch .zshrc or an existing Anaconda)
2. creates the genie env: root lhapdf gsl libxml2 log4cpp cmake make clang gfortran
3. builds Pythia 6.4.28 and TPythia6 into external/install/
4. patches Make.include for Apple Silicon and the conda compilers (idempotent)
5. ./configure --enable-faser ..., make, make install

Options: WITH_PYTHIA8=1, WITH_APFEL=1, NJ=<jobs>.

Building on lxplus (EL9 + CVMFS)

```
git clone https://github.com/rubbiaa/GENIE.git GENIE3.06 && cd GENIE3.06
./build_genie_lcg.sh                # LCG_107, x86_64-el9-gcc13-opt
LCG_VERSION=LCG_105 ./build_genie_lcg.sh # any other release
source setup_lcg.sh
```

- One LCG view provides ROOT, Pythia6, LHAPDF, GSL, log4cpp and libxml2, all built with the same compiler
- `setup_lcg.sh` replaces `ATLAS_container.sh`, `.asetup.save`, `kk` and `setupGenerator.sh`
- Only TPythia6 and GENIE are compiled locally

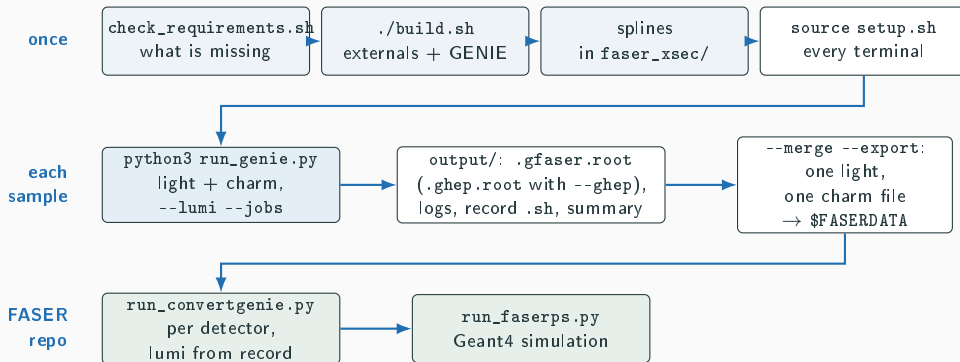
Building on a plain Ubuntu machine

```
sudo apt install build-essential gfortran cmake libxml2-dev \  
                libgsl-dev liblog4cpp5-dev curl  
git clone https://github.com/rubbiaa/GENIE.git GENIE3.06 && cd GENIE3.06  
./check_requirements.sh          # what is there, what is missing  
./build.sh                      # = build_genie_ubuntu.sh  
source setup.sh                 # = setup_ubuntu.sh
```

- No conda, no CVMFS: your own ROOT (ROOT_THISROOT, else newest ~/ROOT/root_install*; a snap ROOT only as last resort)
- ROOT must have geom and mathmore: install `libgsl-dev` *before* building ROOT
- LHAPDF 6 from the system if present, otherwise built into `external/install/`
- Pythia8 found as `~/ROOT/pythia8*` or `PYTHIA8=...`; used with `WITH_PYTHIA8=1`

Running GENIE

Workflow: from checkout to the FASER simulation



Only the middle row is repeated for each new sample. `run_genie.py` replaces the hand-written `faser/run*.sh`, and `gevgen_faser --gfaser` the `convertGHEP.C` pass.

```
source setup.sh
python3 run_genie.py --dry-run    # plan only
python3 run_genie.py             # light + charm, 1000/fb
python3 run_genie.py --jobs 8 --merge --export
python3 run_genie.py --flux light --lumi 2000 \
    --jobs 16 --run 100 --cc-only
python3 run_genie.py --ghep      # also GHEP
```

Files in output/, P = fasercal.Aki2024.v10.light or
.charm:

```
P.0.gfaser.root  P.0.log          (per job)
P.0.ghep.root    (with --ghep)
P.0.status
P.all.gfaser.root      (--merge)
P.r0-7.run_genie.sh    (exact commands)
P.r0-7.run_genie_summary.log (events, s/ev)
```

- **Two samples:** light and charm side by side, own files, seeds, merged file
- **Inputs found** in data/GDML, data/fluxes, faser_xsec
- **Jobs:** luminosity split over --jobs; run $r = \text{--run} + i$, seed = base + r ; one queue, up to one job per core
- **Output:** gFaser only (--no-ghep); --ghep adds GHEP
- **Safe, traceable:** asks before overwriting; records commands and lumi
- **Hand-over:** --export copies P.all.gfaser.root (light, charm), records and summaries to \$FASERDATA/GENIE

Pitfalls solved

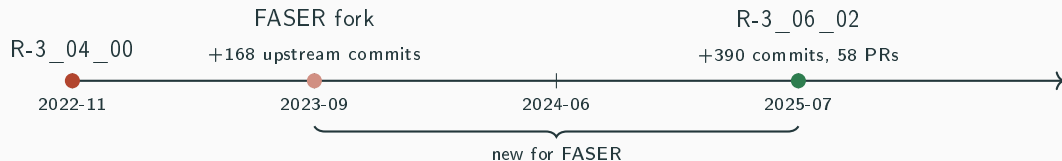
Problems found and fixed (all automatic in the scripts)

Symptom	Cause → fix
conda solve never finishes expected ')' in <charconv>	old Anaconda classic solver → private Miniforge with mamba clang 18 with libc++ 20 headers (ROOT pin) → match clang to the headers
undeclared identifier NAN	macOS 27 SDK takes NAN from <float.h>; strict -std=c++20 hides it → -std=gnu++20
libc++ “feature unavailable” rootcling fails	conda targets macOS 11 → -D_LIBCPP_DISABLE_AVAILABILITY dictionary step does not see CXXFLAGS → flags also passed via the include options
-lPythia6/-lEGPythia6 not found	make distclean empties install/ → externals in external/install/; library path added
APFEL: arm64-apple unknown Apple Silicon flags	2013 config.sub → use aarch64-apple-darwin generic macosx64 block (Xcode-10 era) → dedicated macosxarm64 block

3.04 → 3.06



Where the versions sit



- R-3_06_02 in the FASER GitLab is a plain mirror of the official release: **no FASER code**
- Default tune unchanged: G18_02a_00_000; its model configuration is identical
- About 1 480 files differ upstream (configs, new models, data tables)

FASER code ported to 3.06

Item	Status
gevgen_faser, gmkspl_faser	ported unchanged
FaserGMCJDriver, FaserROOTGeomAnalyzer	ported; base interfaces unchanged
configure --enable-faser, Apps/Makefile	applied cleanly
GVLD-Emax = 7 TeV (official: 1 TeV)	re-applied
NaturalIsotopes::GetIsotopeData()	re-added
weighted-flux POT fix in GSimpleNtpFlux	already upstream
tunes F23_00a, G25_01a	ported
faser/ scripts; geometries and fluxes in top-level data/	copied; big fluxes git-ignored

Not ported: the fork's broken Pythia6 lines in `Make.include`, which also removed the Pythia8 link block.

Pythia6 / Pythia8

- Pythia6 now optional (`--disable-pythia6`)
- decayer: `PythiaDecayer` → `Pythia6Decayer2023` (and `Pythia8Decayer2023`)
- DIS charm: `AGCharm2019` → `AGCharmPythia6Hadro2023` (same parameters)
- HEDIS hadronization: `LeptoHadPythia6/8`
- HE leptons: separate Pythia6/8 W-decay classes

Other

- memory-leak fixes (long productions)
- G_F , α configurable in `CommonParam.xml`
- build: Apple Silicon support, no ROOTSYS needed

Not used by default

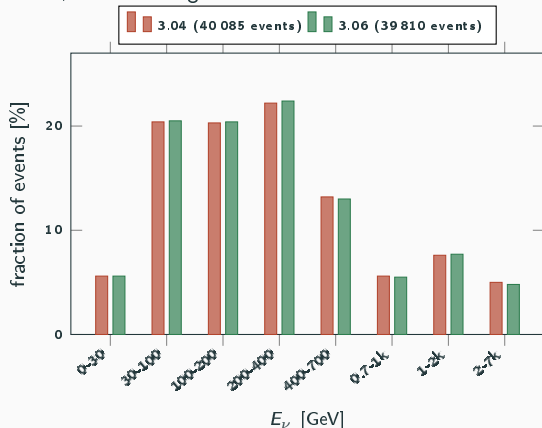
- tunes G24_20i/j/k/l, N24_20i, MK19_00a
- MK single-pion model, Z-expansion and MARun form factors, SuSAv2/CRPA updates

⇒ Expect small event-level differences in decays and charm hadronization: validate before production.

Validation

Event rate and neutrino energy

Same command, flux, geometry and (3.04) splines: `gevgen_faser -l 1000 (1000 fb-1)`, FASERCal V10, Aki 2024 light flux.



- Rate: $3.06/3.04 = 0.993 \pm 0.007$ (1.0σ)
- Every E_ν bin agrees within 1.3σ (ratios 0.96–1.01)
- $\langle E_\nu \rangle = 478$ vs 474 GeV, median 222 vs 222 GeV
- Speed: 0.045 s/event, constant over the run (30 min in total)

	3.04	3.06
$\nu_\mu / \bar{\nu}_\mu / \nu_e / \bar{\nu}_e$ [%]	64.6 / 27.6 / 5.4 / 2.4	65.2 / 27.2 / 5.3 / 2.4
CC / NC [%]	75.6 / 24.4	75.4 / 24.6
DIS / RES / QEL [%]	97.3 / 1.9 / 0.5	97.3 / 1.9 / 0.5
charm production [%]	6.3	6.2
targets Fe / C / Pb / W / Al [%]	76.2 / 7.8 / 7.2 / 3.3 / 2.2	76.0 / 7.8 / 7.2 / 3.3 / 2.3
vertex z , 5–50–95% [m]	0.88 / 5.43 / 6.73	0.87 / 5.42 / 6.73
CC inelasticity $\langle y \rangle$	0.435	0.432
$\langle n_{\text{charged hadrons}} \rangle$	7.87	7.83
$\langle n_{\pi^0} \rangle$	2.40	2.38

Conclusion

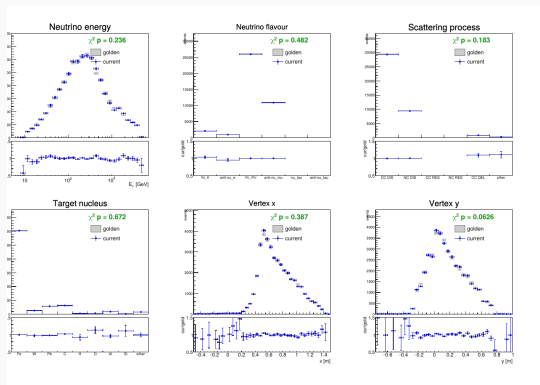
With the same splines, 3.06 reproduces the 3.04 light sample within statistics: rate, spectrum, flavours, processes, targets and final-state multiplicities. The remaining checks are the new 3.06 splines and the charm/ τ samples.

Mac vs Ubuntu: same sample, different events

Same command and seed (1000 fb⁻¹, light, 3.04 splines), Mac (arm64, clang) vs Ubuntu (x86-64, gcc).

	Mac	Ubuntu
events	39 661	39 900
$\langle E_\nu \rangle$ / median [GeV]	476 / 223	472 / 220
ν_μ / $\bar{\nu}_\mu$ / ν_e / $\bar{\nu}_e$ [%]	65.1/27.3/5.1/2.5	65.1/27.3/5.3/2.4
CC [%]	75.8	75.6
DIS / RES / QEL [%]	97.4/1.9/0.4	97.1/2.1/0.5
Fe / C / Pb / W [%]	75.8/7.7/7.3/3.4	75.9/7.9/7.1/3.3
charm events [%]	6.4	6.1
$\langle n_{\text{ch}} \rangle$ / $\langle n_{\pi^0} \rangle$	7.86 / 2.38	7.85 / 2.37

- First 38 events bit-identical, then the event sequences diverge
- Cause: floating-point rounding (clang/arm64 vs gcc/x86); one accept/reject flips, the random sequence shifts
- Statistically identical: rate $+0.8\sigma$, KS $p = 0.37$ (E_ν), 0.20 (n_{ch}), 0.16 (ν_z); regression test: 101/101 PASS



grey band: Mac (golden), points: Ubuntu; lower pads: Ubuntu/Mac

Regression test: run_regression.py

```
python3 run_regression.py --record # golden
python3 run_regression.py # compare
python3 run_regression.py --case charm --quick
python3 run_regression.py --input \
    light=output/<prefix>.0.gfaser.root
```

case	flux	events
light	Aki 2024 light	5000
charm	Aki 2024 charm	5000
nutau	Kling 2021 ν_τ , $\bar{\nu}_\tau$	2000

Golden sample in tests/regression/golden/: JSON (scalars, histograms, health, metadata) + PDF.

- **~100 scalars** with errors: flavours, CC/NC, processes, targets, E_ν , y , Q^2 , W , multiplicities, charm (species, z , p_T , decays), τ (rate, energy, decay modes)
- **~35 histograms** (fixed binning), χ^2 shape test
- **Health** checks must be 0 (NaN, $E < m$, bad indices, ...)
- **Statistical**, since events are exact only on the same binary/platform: WARN at 3σ / $p < 10^{-2}$, FAIL at 5σ / $p < 10^{-4}$; **--exact** for bitwise

Sensitivity (injected into the Ubuntu sample):

vertex $z + 5$ cm	FAIL (v_z shape)
30% fewer D^+ events	FAIL (n_{charm}) + species WARNs
lepton energy +2%	FAIL (y , W , z_c , p_z balance)

Next steps

Next steps

1. **Record the golden sample** (`run_regression.py --record`) and run it after every change
2. **Charm sample:** repeat the 3.04 comparison for the Aki 2024 charm flux
3. **Regenerate splines for 3.06** with `faser/Splines/` (free nucleons $\rightarrow$ nuclei, 6 flavours, up to 7 TeV; $\sim 2-3$ days on 16 cores)
4. **Physics validation** of the rewritten decays and charm hadronization (charm and τ samples)
5. **Optional:** Pythia8 hadronization (`WITH_PYTHIA8=1`), then drop Pythia6
6. **Share:** push `faser-R-3_06_02` back to the FASER GitLab

Backup: command cheat sheet

```
./check_requirements.sh; ./build.sh      # once per machine (build.sh re-checks first)
source setup.sh                          # every terminal: mac, lxplus or ubuntu detected
cd GenieGenerator && make -j && make install && cd ..  # rebuild after a code change

# event generation (FASERCal V10, Aki 2024 light + charm, 1000 fb-1, 8 jobs each) -> output/
python3 run_genie.py --jobs 8 --merge --export      # light + charm -> 2 merged files
python3 run_regression.py                          # compare with the golden sample
# same thing by hand, one job (light):
gevgen_faser -l 1000.0 -r 0 -g $GENIE_HOME/data/GDML/FASERCAL_V10.gdml \
  -f $GENIE_HOME/data/fluxes/Aki_2024/events_light_4x4.root --seed 2999833 \
  --cross-sections $GENIE_HOME/faser_xsec/faserSplines.7TeV.xml \
  -o fasercal.Aki2024.v10.light --gfaser

# splines (run in GENIE3.06/splines_work, one flavour at a time)
source $GENIE/faser/Splines/calcSplinesN.sh; wait
source $GENIE/faser/Splines/mergeSplinesN.sh
source $GENIE/faser/Splines/calcNuMuSplinesA.sh; wait      # ... all 6 flavours
source $GENIE/faser/Splines/mergeSplinesA.sh
```